

Bestagon Design Review

Event processor atomicity and domain event metadata

- **Project:** bestagon — Framework for event-sourcing with Python (DDD + ES + CQRS)
 - **Branch:** v0.8.0
 - **Date:** 2026-09-16
 - **Participants:** Aleksandr Antonov, Claude (Claude Code)
-

Question 1 — Non-atomic event processing

There is a weak point in the code, what can you propose as a solution? Do not change the code.

`_consume_subscription` does `process_event` -> `set_checkpoint` non-atomically, with no idempotency support, no retry, no dead-letter. The library's own comment says "ACHTUNG - dangerous, what if only one operation will be completed???"

Answer

Read through `bestagon/core/event_processor.py:59-80` plus the checkpoint store and both adapters. Here's the analysis and proposal.

Diagnosis: the order is right, the guarantee is missing

`process_event` -> `set_checkpoint` is the *correct* order. The reverse would give at-most-once (checkpoint advances, crash, event never processed — silent data loss, unrecoverable). The current order gives **at-least-once**: crash in the window between the two lines means the event is redelivered on restart.

So the TODO's premise — "should be a single transaction, but there are no transactions in event sourcing" — is only half true. You can't get a distributed transaction between the event store and an arbitrary side effect, and you don't need one. The standard answer is: **at-least-once delivery + idempotent consumers = effectively-once**. Four pieces are missing.

1. Transactional checkpoint for same-store projections (the real fix)

There *are* transactions — just not in the event store. They're on the read side. For a **Projection**, the checkpoint is not infrastructure metadata, it's **part of the read model**. Store it in the same database, in the same transaction as the projection write, and the atomicity problem disappears entirely.

Introduce a unit of work that the processor opens and both sides join:

```
class UnitOfWork(ABC):
    # adapter-specific: neo4j tx, aiosqlite connection
    async def __aenter__(self) -> "UnitOfWork": ...
    async def __aexit__(self, *exc) -> None: ... # commit or rollback

# EventProcessor gains a factory, defaulting to a null Uow:
async with self.begin_unit_of_work() as uow:
    await self.process_event(event=domain_event, uow=uow)
    await self.checkpoint_store.set_checkpoint(checkpoint, uow=uow)
```

`CheckpointStore.set_checkpoint` takes an optional `uow`; when absent it behaves exactly as today, so existing implementations aren't broken.

Both adapters can support this:

- **AIOSQLite** — the UoW is the connection; drop the `commit()` from `set_checkpoint` (aiosqlite_db.py:101) when a UoW is supplied and let the processor commit once. Requires the projection and `_checkpoints` table to live in the same database file (worth asserting at wiring time).
- **Neo4j** — the UoW is an explicit `session.begin_transaction()`. **Caveat:** `Neo4jCheckpointStore` (neo4j_db.py:15) takes its own `database_name`, while `Neo4jProjection` (neo4j_db.py:97) has a separate `get_database_name()`. Neo4j has no cross-database transactions, so this only works if the `Checkpoint` node is written into the projection's own database. That's a wiring constraint the library should enforce rather than leave to the user.

This is the highest-value change: for projections it makes the problem go away rather than mitigating it.

2. Idempotency for everything else

Applications write to the event store, so they can't share the projection's transaction. Two mechanisms, no new infrastructure needed:

Natural dedupe via optimistic concurrency. A redelivered event that re-derives the same command against the same aggregate version will fail `append_events` with `ExpectedVersionError`. Today that's an unhandled crash; it should be recognized as *"already applied, safe to advance the checkpoint."* This requires handlers to derive aggregate IDs deterministically from the source event rather than generating fresh UUIDs — that's a documented contract for handler authors, and the library should state it loudly.

Explicit dedupe key when that's not enough. `StreamEvent` has no `event_id`, but it doesn't need one: `commit_position` is globally unique and monotonic, and `(stream_name, stream_position)` is a stable natural key. A processor writing to a store it controls can record the processed key alongside its write; a `commit_position <= last_seen` check then makes replays free. Worth noting the gap in `DomainEventMetadata` too — adding an `event_id` would make dedupe possible across republishing/copying scenarios where positions change, but it's not required for the crash-recovery case.

3. Retry with error classification

Right now every exception is terminal. That's wrong for a dropped TCP connection and right for a malformed payload. Classify:

Class	Examples	Action
Transient	connection reset, timeout, deadlock, Neo4j <code>TransientError</code>	retry in-place, exponential backoff + jitter, bounded attempts
Poison	<code>TypeNotRegisteredError</code> , JSON decode failure, schema mismatch	no retry — retrying can't help
Concurrency	<code>ExpectedVersionError</code> on replay	treat as already-applied, advance
Unknown	everything else	one conservative retry, then poison

Retry happens *before* the checkpoint advances, so a successful retry needs no special handling. Classification should be overridable per adapter, since only the adapter knows which of its driver's exceptions are transient.

4. Dead-letter and an explicit failure policy

After retries are exhausted, "stop the world" should be a *choice*, not the only behaviour. Make it a per-processor enum, defaulting to today's semantics:

- **STOP** (default) — halt the subscription, preserve ordering, wake a human. Correct for applications and any order-sensitive projection.
- **PARK** — write the raw `StreamEvent` (bytes, not the decoded event — the failure may be the decoding) plus the traceback and checkpoint name to a `DeadLetterStore`, advance the checkpoint, continue. Only legitimate when the handler is order-independent, and the processor should be reported as *degraded* in `SystemHealth` while the DLQ is non-empty.

`DeadLetterStore` mirrors `CheckpointStore`: an ABC with adapter implementations, plus a `replay(processor_name)` operation so a parked event can be reprocessed after a code fix. It pairs naturally with the existing `rebuild_projection` in `system.py:124`.

Two adjacent problems worth fixing at the same time

The failure is invisible. On the error path the exception is re-raised inside `_subscription_task`, which nobody awaits until `stop()` is called. Python will emit "Task exception was never retrieved" at GC and the actual traceback may never surface. The task needs a done-callback that records the exception on the processor and surfaces it through `SystemHealth`. Relatedly, `running` returns `True` when `subscription` is `None` (`event_processor.py:32-34`) — a never-started processor reports healthy.

`logger.warning` for a critical failure (`event_processor.py:77`) should be `logger.error` — this is the line that tells an operator the system just stopped.

Bonus once idempotency exists: the per-event checkpoint write becomes optional. You can checkpoint every N events or T milliseconds and trade a larger replay window for significantly less write amplification — safe precisely *because* handlers are idempotent. Not a fix, but it's the payoff.

Suggested sequence

- 1 Retry + error classification — self-contained, no interface change, immediate value.
- 2 Failure policy enum + `DeadLetterStore` — new ABC, **STOP** default keeps behaviour identical.
- 3 `UnitOfWork` threaded through `set_checkpoint` and `process_event` — the structural fix; optional param keeps it backward-compatible.
- 4 Document the idempotency contract for handler authors and handle `ExpectedVersionError`-as-replay.

Steps 1-2 make failures survivable; step 3 makes the original TODO obsolete for projections; step 4 covers the rest.

Question 2 — Flexible domain event metadata

One more thing - take a look at `DomainEventMetadata` class it is currently a frozen dataclass, what I want is to make it more flexible - to enforce necessary fields, but also give a possibility to add arbitrary fields when necessary, any suggestions how to make it?

Answer

Looked at the class and every call site. Before the "how" — there's a bug hiding in the current design that the same change fixes.

The closed class isn't just inflexible, it breaks on read

`mapper.py:97` does:

```
metadata = DomainEventMetadata(**metadata_dict)
```

Any key in the stored JSON that isn't one of the four fields raises `TypeError`. Events are immutable and kept forever, so this is a **forward-compatibility failure**: the moment one service writes events with an extra metadata field — or you add a field in v0.9 and a v0.8 consumer is still running — reads crash on historical data. Metadata in an event store must be *open for reading* as a matter of correctness, independent of whether you want user-defined fields.

So "allow arbitrary fields" and "don't explode on unknown fields" are the same fix.

Recommendation: frozen pydantic model with typed extras

Pydantic is already a hard dependency (`pydantic>=2.13.4`, and `NewStreamEvent/Checkpoint` already use it), so this adds nothing and makes metadata consistent with the rest of the core:

```
from pydantic import BaseModel, ConfigDict, JsonValue

class DomainEventMetadata(BaseModel):
    model_config = ConfigDict(extra='allow', frozen=True, strict=True)

    # Extras are validated as JSON-representable, so a bad value fails
    # at construction rather than deep inside append_events.
    __pydantic_extra__: dict[str, JsonValue]

    timestamp: str
    aggregate_id: str
    aggregate_version: int
    aggregate_type: str

    @staticmethod
    def create_timestamp() -> str:
        return datetime.now(timezone.utc).isoformat()
```

That single config line buys four things at once:

- `extra='allow'` — arbitrary fields accepted at construction *and* on deserialization, reachable by normal attribute access (`metadata.tenant_id`), and included in `model_dump()`. Round-trip is lossless, which is what kills the read bug above.
- `frozen=True` — preserves today's immutability, and events stay value objects.
- **Required fields actually enforced** — right now `aggregate_version="3"` sails straight through the plain dataclass and corrupts the version arithmetic in `repository.py:89-90`. Pydantic rejects it. You gain validation you don't currently have.
- `__pydantic_extra__: dict[str, JsonValue]` — the part most worth insisting on. Without it extras are `Any`, and someone putting a `Decimal` or a custom object in metadata gets a `TypeError` from `json.dumps` at `mapper.py:106` — far from the cause. Annotating the extras dict moves that failure to the construction site, with a real field name in the message.

One guard worth adding: extras land flat in the JSON alongside the required fields, so a user field named `timestamp` — or one that collides with a field *you* add in a future version — is a silent semantic break. A model validator that rejects a small reserved set (`event_id`, `correlation_id`, `causation_id`, ...) keeps the namespace evolvable without forcing extras into a nested dict.

Knock-on changes

Serialization — `mapper.py:106 asdict(domain_event.metadata)` stops working; it becomes `domain_event.metadata.model_dump(mode='json')`, and line 97 becomes `model_validate(metadata_dict)`.

`get_payload` still works — `aggregate.py:31` calls `asdict(self)`, which deep-copies the non-dataclass metadata and then discards it at the `pop`. Correct, just wasted work; a dict comprehension over `fields(self)` would be cheaper.

Hashability caveat — `DomainEvent` is a frozen dataclass, so it generates `__hash__`. A frozen pydantic model is hashable only if its values are, and `JsonValue` permits lists and dicts. Hashing an event whose metadata holds a nested extra will raise. Worth a docstring line; only bites if you put events in sets.

The other half: a pluggable metadata class

`extra='allow'` covers per-event ad-hoc fields. It does *not* cover "our whole system requires `tenant_id` on every event" — extras are optional by definition, so nothing enforces it.

This is exactly the TODO at `mapper.py:94` ("*Metadata class is hardcoded... is there a possibility to use another class?*"). Both needs are served if `Mapper` holds the metadata class instead of hardcoding it:

```
mapper.register_metadata_class(MyMetadata)    # subclass of DomainEventMetadata
```

`to_domain_event` then validates against the registered class. A team subclasses to make their fields **required and typed**, and still gets `extra='allow'` inherited for the genuinely one-off cases. Enforced *and* open, which is what was asked for — the two mechanisms are complementary, not competing.

Why not the alternatives

Approach	Why not
<code>extra: dict = field(default_factory=dict)</code> on the dataclass	No collision risk, but no attribute access, no validation of extras, and it doesn't fix the read bug — unknown <i>top-level</i> keys still raise. You'd also have a mutable dict on a frozen dataclass, silently breaking <code>__hash__</code> .
Dataclass + <code>__init__(**kwargs)</code> stashed on the instance	Hand-rolling what pydantic already does, and loses frozen .
<code>TypedDict</code> / plain dict metadata	Maximum flexibility, zero enforcement — you asked for both.

While you're in here

Metadata is where the previous discussion dead-ended: there's no `event_id`, which is what dedupe wants when `commit_position` isn't stable. If you're changing this class anyway, that's the moment to promote `event_id`, `correlation_id`, and `causation_id` from hypothetical extras to required fields. Causation/correlation are the standard pair for tracing a command through the events it caused, and every serious ES system ends up needing them. Adding them now costs a field; adding them at v1.0 costs an upcaster for every event ever written.

That does raise migration: existing stored events lack those keys, so they'd have to be `None`-defaulted, or you introduce the upcaster hook now. Defaulting them to `None` at this stage is the pragmatic call — the project is at **Development Status :: 3 - Alpha**, and it's much cheaper than the alternative later.

Summary of open proposals

#	Proposal	Area	Status
1	<code>UnitOfWork</code> shared by <code>process_event</code> and <code>set_checkpoint</code>	event_processor, checkpoint_store, adapters	Proposed
2	Idempotency contract + <code>ExpectedVersionError</code> -as-replay	event_processor, docs	Proposed
3	Retry with transient/poison error classification	event_processor, adapters	Proposed
4	<code>DeadLetterStore</code> + <code>STOP/PARK</code> failure policy	new ABC + adapters	Proposed
5	Surface subscription task failure via <code>SystemHealth</code>	event_processor, system	Proposed
6	<code>DomainEventMetadata</code> as frozen pydantic model, <code>extra='allow'</code>	aggregate, mapper	Proposed
7	Pluggable metadata class on <code>Mapper</code>	mapper	Proposed
8	Add <code>event_id</code> / <code>correlation_id</code> / <code>causation_id</code>	aggregate	Proposed

No code was modified during this conversation.